Modular-RAG Azure

A plain-English project report

What we use, why we use it, and how the complete application works.

Prepared: 18 October 2026

For: Readers with no programming or cloud background

Based on: The application code, interface, tests, and evaluation files in this project.

A guide to document upload, four search methods, AI answers, source checking, storage, testing, and current limitations.

Contents

1. What is this project? 3
2. What does the project name mean? 3
3. What problem does it solve? 4
4. The whole system at a glance 4
5. Everything we use and why 5
6. Why there are several places to store information 6
7. What happens when a PDF is uploaded? 7
8. The four ways the application searches 8
9. How the knowledge graph is built 9
10. What makes the system agentic? 10
11. How the answer is checked 10
12. Keeping questions tied to the selected document 11
13. What the user sees 11
14. The backend and its main actions 12
15. Document status and failure handling 12
16. Supporting building blocks explained 13
17. Security and privacy: what is present and what is missing 13
18. How the project is tested 14
19. How retrieval quality is evaluated 14
20. What affects speed and cost? 15
21. Current limitations in simple language 15
22. An example from beginning to end 16
23. Where the main work lives in the project 16
24. Short glossary 18
25. A short explanation for a presentation 19

This report explains the existing document-question-answering application. The recent Docker preparation work is outside the scope of this report. Examples are illustrations, not answers generated from a real customer document. Reasons given for technology choices explain how those choices help this implementation; they are not a record of the original designer's intentions.

1. What is this project?

Modular-RAG Azure is an AI assistant that helps people find answers inside PDF documents.

A person uploads a PDF, selects it, and asks a question in everyday language. The application searches that document for useful information, prepares an answer, adds source references, and checks whether the answer is supported by the retrieved text.

For example, someone could upload an employee handbook and ask, “What does the document say about annual leave?” The application should find the relevant passages and answer using those passages. It should avoid inventing a policy when the document does not provide enough information.

The main purpose is to make documents easier to use while giving the reader a way to inspect the evidence behind an answer.

2. What does the project name mean?

Part of the name	Simple meaning	Why it matters
Modular	The application is divided into smaller parts with separate jobs.	A search method or checking step can be improved without rewriting the whole application.
Retrieval	Finding relevant information in the selected document.	The system needs useful evidence before it writes an answer.
Augmented	Giving the AI extra information for the current question.	The AI receives document passages instead of relying only on what it learned earlier.
Generation	Writing an answer in readable language.	People receive an explanation rather than only a list of matching passages.
Azure	Microsoft's cloud platform.	Several storage, search, and AI services used by the project are hosted there.
Agentic	The application chooses a search method and can try another method when needed.	Different types of questions benefit from different search approaches.

RAG stands for **Retrieval-Augmented Generation**. An everyday comparison is an open-book examination: first find the relevant pages, then answer using those pages.

Uploading a document does not train a new AI model. The project prepares the document for searching and supplies selected passages to an existing AI model when answering questions.

3. What problem does it solve?

Long documents can take time to read. People may not know the exact words to search for. Useful information may be spread across several passages. An ordinary AI answer may sound convincing even when it is not supported by the user's document.

This project addresses those problems by combining document storage, several search methods, answer generation, and evidence checks.

Possible uses include understanding manuals, reviewing training material, exploring research reports, and answering questions about internal reference documents. These are potential uses of the design, not a claim that every document type has been evaluated.

4. The whole system at a glance

The application has two main journeys.

Preparing a document:

PDF upload → read its text → divide it into small passages → save the original file → create numerical representations of the passages → store searchable records → register the document as ready.

Answering a question:

Select a document → ask a question → choose a search method → collect evidence → check whether it is enough → write an answer → check references → check factual claims → display a verified answer or a refusal.

A separate, optional action builds a relationship map for Graph RAG. The document can already be searched before that map is ready.

5. Everything we use and why

Technology or component	What it does in this project	Why it is useful
Python	The programming language used for the application.	It connects the interface, AI services, document processing, and databases using one language.
Streamlit	Creates the screen the user interacts with.	It provides upload controls, a document selector, chat messages, and evidence displays without requiring a large custom interface.
FastAPI	Receives requests and coordinates the work behind the screen.	It gives the interface a clear way to request uploads, document lists, answers, and graph builds.
Uvicorn	Runs the FastAPI application as a web service.	FastAPI defines the application's actions; Uvicorn accepts the incoming web connections.
Pydantic	Checks the format of incoming and outgoing information.	It catches problems such as an invalid document identifier or an excessively long question.
Requests	Lets the Streamlit interface communicate with the backend.	It carries questions and uploads to the application and brings responses back.
pypdf	Reads text from PDF pages.	The search system needs text it can process, not just a file attachment.
Azure Blob Storage	Stores the original PDF.	It keeps the source document separately from the smaller passages used for searching.
Microsoft Foundry model deployments	Provide access to the configured chat and embedding models.	The project can use hosted AI models instead of running large models on the user's computer.
Chat model	Chooses search tools, creates query variations, extracts relationships, writes answers, and evaluates evidence.	These tasks involve understanding and producing language.
Embedding model	Converts text into a list of numbers that represents aspects of its meaning.	It allows meaning-based search even when a question and passage use different words.
OpenAI Python client library	Sends model requests to the configured Azure endpoint.	It provides the application's connection to the chat and embedding services. Its name does not mean this application sends requests to the public ChatGPT website.
Azure AI Search	Stores and searches document passages and their numerical representations.	It supports both keyword and meaning-based retrieval.
Neo4j AuraDB	Hosts the relationship database used for Graph RAG.	It stores connections between concepts, facts, and supporting document passages.
SQLite	Keeps the local document register.	A small local database is enough to track document names, identifiers, readiness, and graph status at this stage.

Technology or component	What it does in this project	Why it is useful
Azure Identity library	Helps the application authenticate with Azure.	It allows the application to prove which identity is making a request.
Azure CLI login	Supplies the existing local development identity by default.	The developer can use an Azure login for the services that accept this authentication method.
Environment variables and .env	Hold settings such as service addresses, deployment names, and credentials.	The same code can use different configured services without embedding those details in the source code.
python-dotenv	Reads the local .env settings file.	It makes those settings available to the application during local development.
pytest and FastAPI TestClient	Run automated checks.	They help catch mistakes after changes without manually testing every behavior.
JSON	Carries structured information between components and in some AI responses.	Named fields make information easier for the application to check and use.
CSV	Stores the retrieval evaluation results in a table-like file.	Results can be reviewed and compared outside the running application.

The specific chat and embedding model deployments are supplied through configuration. The source code alone is not a reliable basis for claiming a particular model family or model size.

6. Why there are several places to store information

These stores have different jobs. They are not interchangeable copies of one database.

Storage location	Information kept there	Everyday comparison
Azure Blob Storage	The original uploaded PDF.	A filing cabinet holding the original document.
Azure AI Search	Passages, their numerical representations, file names, page numbers, and identifiers.	A detailed searchable index of the filing cabinet.
Neo4j AuraDB	Concepts, candidate facts, relationships, and links back to supporting passages.	A map showing which ideas are connected and where those connections came from.
Local SQLite registry	Which documents the application knows about and what stage each has reached.	A reception register showing which files are available and ready.
Streamlit session state	The current interface session's displayed chat histories and selection.	Temporary notes for the current visit.

The local document register is not a backup of the PDFs or the cloud search index. Preserving only that database would not preserve the whole system.

7. What happens when a PDF is uploaded?

7.1 The upload is checked

The API accepts PDF uploads up to 10 MiB, shown to users as a 10 MB limit. It checks the filename, rejects empty files, and checks for the expected PDF header. It cleans the filename so that unusual characters or folder paths do not become unsafe storage names.

These checks catch common invalid inputs. They are not a complete malware scan or a guarantee that every file is harmless.

7.2 The text is read page by page

The pypdf library extracts the available text. Page numbers are kept so that an answer can later tell the reader where its evidence came from.

The current implementation expects text-based PDFs. A scanned page may only be a picture of words. Reading those words needs OCR - optical character recognition - which is not implemented in this upload flow. A PDF with no extractable text is rejected.

7.3 Text is divided into chunks

A chunk is a small passage of text. The implementation uses passages of up to 1,000 characters, with 150 characters shared between neighboring passages on the same page.

The shared part is called overlap. It helps preserve context at a boundary. For example, a sentence beginning at the end of one passage may still have useful surrounding text in the next passage.

Characters are letters, spaces, and punctuation - not words. This method is simple, but it can split sentences or tables awkwardly.

7.4 The document gets a unique identifier

The application gives the upload a UUID, which is a long identifier designed to be unique. Passages receive identifiers connected to that document.

This allows two files with the same name to remain separate. Uploading the same PDF again creates a new document; the code does not automatically recognize it as an existing upload.

7.5 The original is stored

The PDF is uploaded to Azure Blob Storage under a path containing its document identifier. This helps prevent one same-named file from replacing another.

7.6 Each passage gets an embedding

The embedding model turns each passage into a list of numbers. In this implementation, the list must contain 1,536 numbers.

Imagine assigning a passage a position on a very detailed map of meaning. A question can also be placed on that map, making nearby passages possible matches.

The number 1,536 is a compatibility requirement in the current code. It is not a percentage accuracy score, and it does not by itself identify the model being used.

7.7 Searchable records are uploaded

The application sends records to Azure AI Search in batches of up to ten. Each record includes the passage, its embedding, the document identifier, filename, and page number.

The upload results are checked for failures. The document is registered as searchable only after ingestion finishes successfully.

There is no complete automatic rollback across cloud services. A failure after the original PDF or some passages were stored can leave partial data behind for later cleanup.

8. The four ways the application searches

8.1 Vector RAG: search by meaning

Vector search compares the numerical representation of the question with those of the document passages.

For example, “How can I recover access?” may match a passage about resetting a password even if the wording differs.

Why used: People naturally ask questions in their own words. This method helps find related meaning.

Limit: A passage can be similar in meaning without fully answering the question. Similarity is not proof.

8.2 Hybrid RAG: search by words and meaning together

Hybrid search combines the question's actual words with its numerical representation in Azure AI Search.

For example, a question mentioning a particular policy code benefits from matching that code, while meaning-based search helps interpret the rest of the question.

Why used: Some questions depend on exact names, abbreviations, or terminology as well as general meaning.

Limit: Matching a term does not guarantee that the surrounding passage contains the required answer.

8.3 RAG Fusion: ask the search system in several ways

The chat model generates alternative search queries. The code keeps the original question and includes up to three alternatives, removing duplicates.

Each query searches the selected document. The results are combined using Reciprocal Rank Fusion, or RRF.

RRF rewards passages that rank well across the result lists. Think of comparing several reading lists and giving more weight to passages that repeatedly appear near the top.

Why used: A broad question may have several aspects, and one wording may miss useful information.

Limit: Extra queries mean extra model and search work. The AI may also produce an unhelpful variation. If the generated query format cannot be read, the code falls back to the original question.

8.4 Graph RAG: search relationships

Graph RAG looks for connections between concepts in Neo4j.

For example, a passage might say, “The planning component selects the next action.” A stored relationship could connect “planning component” to “next action” through “selects.” The original quote remains linked to that relationship.

Why used: Some questions ask how things relate, rather than simply asking for a definition.

Limit: The current graph search finds matching concepts and source-linked facts. It is not an unlimited reasoning engine, and a relationship extracted by AI can still be wrong.

Type of question	Intended first choice
A general meaning or explanation question	Vector
A question containing exact terminology	Hybrid
A broad question with several aspects	Fusion
A question about relationships between concepts	Graph

These are routing guidelines, not guarantees that one method will always be best.

9. How the knowledge graph is built

The user chooses “Build knowledge graph” for a searchable document. The application loads that document's indexed passages and creates a document-and-passage structure in Neo4j.

The AI then proposes facts from each passage. A proposed fact contains a subject, a relationship, an object, and a supporting quote.

For example:

- Subject: planning component.
- Relationship: selects.
- Object: next action.
- Evidence: the exact sentence from the source passage.

The code checks the fields and requires the quote to occur in the original passage. It rejects malformed entries, excessively long names, and other invalid cases. Accepted entries are stored as **candidate facts**, preserving the distinction between extraction and proven truth.

The graph includes four main kinds of item: documents, passages, entities or concepts, and facts. Connections show which passages belong to a document, which entities a fact concerns, and which passage supports a fact.

Unique identifiers and database constraints help prevent duplicate graph items. Repeat operations can reuse existing items rather than blindly adding new copies.

The main graph-building flow checks that the number of copied passages matches the number loaded from search. It currently requests up to 1,000 search passages, so larger-document handling needs further review.

10. What makes the system agentic?

The agent is the part that coordinates retrieval. It is not a separate person or an unrestricted autonomous program.

First, the chat model chooses a search tool. If that choice is invalid or the routing request fails, the application uses hybrid search as its default fallback.

After a search, the application asks whether the collected evidence is sufficient to answer the question. If not, it can try another search method.

The selected method runs first. The remaining order follows hybrid, fusion, graph, and vector, skipping any method already attempted. A method is not run twice in the same retrieval sequence, and there are at most four retrieval attempts.

Evidence from earlier attempts is kept and combined, with duplicates removed. Therefore, the final answer can use evidence collected by more than one tool. The displayed “successful tool” is the method used at the point the accumulated evidence was judged sufficient; it does not necessarily mean that every source came from that tool.

This design limits repeated work and allows one failed or unhelpful search method to be followed by another.

11. How the answer is checked

Check 1: Is there enough evidence?

Before writing the answer, the chat model evaluates whether the retrieved passages support a complete response. If the passages are missing or judged insufficient, the agent searches again or returns a refusal.

This is an AI judgment. It can make mistakes, but it provides an additional check before generation.

Check 2: Do the citations exist?

A citation such as [1] points to evidence item number one.

The citation validator checks whether the answer contains source references and whether their numbers refer to actual evidence items. If only three items exist, [9] is invalid.

This checks the reference structure. It does not establish that the referenced text supports the sentence.

Check 3: Does the evidence support the claims?

Groundedness means that the answer is supported by the supplied evidence.

The evaluator is instructed to identify meaningful factual claims and assess each one against its cited passages. Unsupported statements, missing citations, contradictions, and unsupported extensions should fail this check.

For example, suppose a document says a benefit applies after six months. An answer saying it applies immediately should be rejected even if it cites a real passage.

The evaluator's response format is also checked. An error or invalid response does not count as a pass.

Check 4: Can the answer be corrected?

Answer generation allows a retry when citation structure fails. The final verification layer separately allows one correction attempt when citation or groundedness checks fail.

If verification still fails, the application returns a refusal rather than displaying the rejected answer as reliable. It also removes supporting-evidence output from that failed result.

What a pass actually means

A “grounded” result means the implemented checks accepted the answer. It is not a guarantee of perfect accuracy. The evaluator itself is an AI model, may miss claims, and uses the same configured chat deployment as other language tasks. Also, a source document can contain incorrect or outdated information.

12. Keeping questions tied to the selected document

When the user selects a PDF, the interface sends that document's identifier with the question. The backend validates the identifier and checks whether the document is registered and searchable.

The selected identifier is temporarily attached to that request. Every retrieval method uses it to restrict searching to the selected document. After the request finishes, that temporary selection is reset.

An everyday comparison is attaching a file number to an enquiry so that every person handling it consults the same file.

This reduces accidental mixing of evidence between documents. It is not user authorization: checking which document is selected is different from checking whether a particular person is allowed to read it.

The code also retains a default sample-document identifier for older scripts and requests that omit a document selection.

13. What the user sees

The Streamlit interface provides:

- A PDF upload control and an upload-and-index action.
- A document library with names, shortened identifiers, and processing status.
- A way to select the active document.
- A graph-building action when appropriate.
- A refresh action to obtain updated document information.
- A chat area for questions and answers.
- A visible groundedness result and the successful retrieval tool.
- Expandable evidence showing the source text, filename, page, and search method.
- Expandable retrieval and verification records.
- Separate displayed chat histories for different selected documents.
- A clear-chat action for the current document's history.

The displayed history is held in the interface session. The current question request does not send that full history to the backend. Therefore, a follow-up such as “Explain that more” should not be assumed to carry the earlier conversation into the answer-generation process.

14. The backend and its main actions

An API is an agreed way for one program to ask another program to do something. Here, the interface asks the backend to perform these actions:

Action	What happens
Application information	Returns the application's name, status, and version.
Health check	Confirms that the backend can respond. It does not test all external services.
Upload document	Validates a PDF, processes it, and records it after successful indexing.
List documents	Returns the locally registered document library.
Get document details	Returns information about one document.
Ask a question	Runs document selection, retrieval, answer generation, and verification.
Build graph	Builds the selected document's relationship data and updates its status.

Questions must be between 3 and 2,000 characters long. Invalid identifiers and documents that are not ready are rejected with appropriate responses.

Some operations take longer because they call cloud services. Upload and graph-building work is moved to worker threads so that this blocking work does not occupy the main asynchronous request-handling path. This is not a durable background-job system: the request still waits for the work to complete.

15. Document status and failure handling

Status	Meaning for a nontechnical reader
search_ready	The document has been indexed and can be searched.
graph_processing	Its relationship map is being built. Ordinary searching remains available.
graph_ready	The graph-building process completed. This does not guarantee that every possible relationship was extracted.
graph_failed	Graph building failed, but the document can still be searched using the existing search index.

The database claims a graph build before starting it. This helps prevent two requests from starting the same build at the same time. A graph already marked ready is not rebuilt through the normal action.

When a retrieval method fails, the agent records the problem and can continue with another method. When verification fails, the answer is not marked grounded. When the interface cannot reach the backend, it shows a readable error.

Some recovery remains manual. For example, a process crash during a graph build could leave a document marked as processing because there is no durable job-recovery service in the inspected implementation.

16. Supporting building blocks explained

Several smaller components help the main features work reliably:

Building block	Why the project uses it
File paths	Find the project's files consistently and locate the document registry.
Temporary folders	Hold uploaded PDFs while ingestion runs, then remove those temporary local copies.
Regular expressions	Recognize citation numbers and clean filenames.
UUIDs	Give each document a distinct identity.
Context variables	Keep the selected document attached to the current request.
Data classes	Keep related information, such as answer text and evidence, together in predictable structures.
Text normalization	Make entity names more consistent before graph matching and storage.
Hashing	Create repeatable identifiers for extracted graph facts. This is not encryption of the facts.
SQL	Read and update the small SQLite document register.
Cypher	Ask Neo4j questions about documents, facts, and relationships.
Parameterized database queries	Pass identifiers and values separately from query instructions, reducing unsafe string handling.
Timeouts	Limit how long the interface waits for requests. A client timeout does not necessarily cancel work already running on the server.
Logs and traces	Record what was attempted and why a result passed or failed.
Virtual environment	Keeps this project's Python packages separate from other Python projects.
.gitignore	Lists local files, such as credentials and generated databases, that should normally stay out of version control. It does not encrypt them.

17. Security and privacy: what is present and what is missing

The inspected application includes file validation, filename cleaning, document identifier validation, document-specific retrieval, parameterized database queries, and restricted browser-origin settings for local development.

Prompts tell the AI to treat retrieved document text as data rather than instructions. This matters because a document could contain text such as “Ignore previous instructions.” Those instructions help define intended behavior but do not guarantee complete protection from malicious documents.

The system uses cloud services. The original PDF is sent to Blob Storage. Passage text is sent for embeddings and stored in Search. Questions and selected evidence are sent to the configured chat model. Graph building also sends passages for fact extraction and stores information in Neo4j. This is not an entirely offline application.

The inspected API does not implement end-user login, per-user document ownership checks, or request-rate limiting. Browser-origin restrictions are not a replacement for those controls. These gaps matter before exposing the application to a wider audience.

Credentials and private endpoint details are intentionally not reproduced in this report.

18. How the project is tested

Automated tests check specific expected behaviors. They act like a repeatable checklist that runs after code changes.

The existing 32 application tests cover areas including citation validation, agent selection and fallback, refusing unsupported answers, final verification, document-specific searching, and API behavior. The API checks include health, document listing, invalid input, selected-document handling, and avoiding an unnecessary graph rebuild.

During this session, all 35 tests in the working folder passed. That total includes the existing 32 application tests and three authentication-selection tests added during the earlier work. The latter are outside this report's application scope. The successful run used a one-thread setting for numerical libraries after memory failures were observed on the machine.

Many tests replace external services with controlled sample responses. This makes tests repeatable and avoids relying on live cloud services for every check.

Passing these tests does not establish that every Azure permission is correct, every real PDF works, or the system can handle a large number of simultaneous users. Those require separate live and load testing.

19. How retrieval quality is evaluated

The project includes five sample questions and expected relevant passage identifiers. An evaluation script compares vector, hybrid, fusion, and graph retrieval using those questions.

It records the time taken, whether a search succeeded, which passages were returned, and retrieval-quality measures where relevance labels are supplied.

Measure	Simple explanation
Precision at five	Of the five result positions being evaluated, how many contain passages marked useful?
Recall at five	Of the passages marked useful for that question, how many were found in the first five results?
Reciprocal rank / MRR	How close to the top was the first useful result? The average across questions is called mean reciprocal rank.
Latency	How long the search took.

For example, if three of the first five results are labeled useful, precision at five is 3 out of 5. If there are six useful passages overall and those three were found, recall is 3 out of 6.

The result file is useful for comparison, but this report does not claim an overall accuracy percentage. A small question set about one sample document is not enough to establish performance across many subjects and document formats. The correctness of the relevance labels also needs human review.

20. What affects speed and cost?

The project uses hosted models instead of running large language models locally. The computer still runs the interface, backend, file processing, and local register.

Uploading a longer document creates more passages, leading to more embedding calls and more stored records. Building a graph adds model calls for extracting facts. A difficult question may cause multiple searches, evidence evaluations, answer attempts, and a final groundedness check.

Fusion usually requires more work than a single search because it generates and executes additional queries. Repeated fallback and correction also increase work.

The four-attempt retrieval limit, duplicate removal, small indexing batches, and bounded answer correction help control unnecessary activity. They do not establish a fixed response time or fixed price.

Actual charges depend on the configured services, models, service tiers, and usage. No billing data was examined for this report, so no exact cost or savings figure is claimed.

21. Current limitations in simple language

1. **Scanned PDFs need another capability.** The current reader extracts existing text; it does not perform OCR.
2. **Layout can be lost.** Tables, columns, and complex formatting may not become clean text.
3. **AI checks are imperfect.** A groundedness pass is useful evidence of checking, not proof of truth.
4. **The source can be wrong.** The application checks against document evidence, not against all real-world facts.
5. **Questions are document-specific.** The normal interface selects one document at a time; it does not perform a combined search across the entire library.
6. **Displayed chat history is not full conversational memory.** Earlier messages are not included in the current backend question request.
7. **Cloud availability matters.** Internet connectivity, credentials, service limits, and outages can affect results.
8. **Identity and access controls are incomplete for public use.** There is no implemented end-user access-control layer in the inspected API.
9. **The registry is local.** Moving the application without its registry can lose the document listing even if cloud data remains.
10. **Processing is not a durable background workflow.** Long work and recovery after crashes need additional design for a larger service.
11. **Partial uploads may need cleanup.** Storage and indexing do not share a single all-or-nothing transaction.
12. **Deletion is not provided through the current API.** A coordinated document-deletion lifecycle across all stores is not implemented.
13. **Graph extraction has practical limits.** It uses model judgments and a bounded passage-loading request.
14. **Tests cover selected cases.** A broad production-readiness, security, and performance assessment has not been established by the unit tests.

These limitations define the current scope. They do not erase the useful document search and verification functionality already implemented.

22. An example from beginning to end

Imagine a user uploads a text-based equipment manual and asks, “What checks are required before starting the machine?”

1. The application validates the file and reads its pages.
2. It divides the text into passages and keeps their page numbers.
3. It stores the PDF and prepares the searchable passages.
4. The manual appears in the document library.
5. The user selects that manual and asks the question.
6. The agent chooses a retrieval method, perhaps hybrid search for the manual's terminology.
7. The search returns passages only from the selected manual.
8. If the evidence is incomplete, another method may be tried and its evidence added.
9. The AI writes an answer with numbered references.
10. The application checks the reference numbers and asks whether the claims are supported.
11. If accepted, the answer and source passages appear in the interface.
12. If the answer cannot be verified after correction, the user receives a refusal instead.

The intended benefit is a faster route to relevant instructions with visible evidence. The example does not imply that this application has been validated for safety-critical equipment decisions.

23. Where the main work lives in the project

This map is included for anyone who later needs to discuss the project with a developer. No programming knowledge is needed to understand the responsibilities.

File or folder	Responsibility
frontend/streamlit_app.py	The user screen, document controls, chat, and evidence display.
app/api/main.py	The backend's available actions and request coordination.
app/api/schemas.py	Rules for the information entering and leaving the question API.
app/api/document_registry.py	The local document register and processing status changes.
app/ingestion/pdf_ingestion.py	PDF reading, passage creation, embeddings, Blob upload, and indexing.
app/retrieval/standard_rag.py	Meaning-based search.
app/retrieval/hybrid_rag.py	Combined keyword and meaning-based search.
app/retrieval/rag_fusion.py	Alternative queries and merged rankings.
app/retrieval/document_context.py	Keeping a search tied to the selected document.
app/graph/build_graph_schema.py	Creating the graph structure for documents and passages.

File or folder	Responsibility
app/graph/extract_knowledge.py	Extracting, checking, and storing candidate relationships.
app/graph/document_graph_service.py	Coordinating graph building for one document.
app/graph/graph_rag.py	Finding graph facts and their supporting quotes.
app/agents/agent_controller.py	Choosing retrieval methods, collecting evidence, and generating answers.
app/agents/citation_validator.py	Checking that source reference numbers exist.
app/agents/groundedness_evaluator.py	Asking whether answer claims are supported by evidence.
app/agents/verified_agent.py	Final checking, correction, and refusal behavior.
tests/	Automated checks of expected behavior.
scripts/evaluate_retrieval.py	Comparing retrieval methods on sample questions.
Other service-test scripts	Development checks for embeddings, search, Neo4j, and groundedness. These can involve live services.
evaluation/	Sample evaluation questions and saved retrieval results.
data/	Local sample documents and the SQLite registry.

24. Short glossary

Word	Everyday meaning
Backend	The part doing the work behind the user screen.
Frontend	The screen and controls the user sees.
Cloud	Computing and storage services accessed over a network.
Model	An AI system trained to perform tasks such as understanding or generating language.
Deployment	A configured, available instance of a model service that the application can call.
Endpoint	The address used to contact a service.
Credential	Information used to prove an application's identity or access rights.
Ingestion	Preparing an uploaded document for storage and search.
Chunk	A small passage taken from a document.
Metadata	Descriptive details such as filename and page number.
Embedding or vector	A list of numbers used to represent aspects of text meaning.
Index	An organized collection that makes information searchable.
Retrieval	Finding potentially useful information.
Top-k	The requested number of leading search results; the agent commonly requests five per tool.
Entity	A named thing or concept, such as a person, process, or component.
Knowledge graph	Stored items and the connections between them.
Evidence	Source text used to support an answer.
Citation	A reference pointing to evidence.
Claim	A factual statement in an answer.
Groundedness	Whether an answer is supported by the supplied evidence.
Fallback	Trying another approach when the first one is insufficient or fails.
Trace	A record of the steps the application took and their outcomes.
Hallucination	An AI-generated statement that is invented or unsupported.
Fail closed	Refusing to treat a result as verified when checking fails.

25. A short explanation for a presentation

“Modular-RAG Azure is a document-based AI assistant. A user uploads a PDF and asks a question. The system searches the selected document using meaning, keywords, alternative queries, or stored relationships. It then writes an answer with references and checks whether the claims are supported by the document. If it cannot verify the answer, it says so. Azure provides storage, search, and AI services; Neo4j stores relationships; FastAPI coordinates the work; and Streamlit provides the user screen.”